Responsible AI and confidential inferencing - NetApp AI with Protopia Image Transformation

NetApp Solutions

NetApp
March 19, 2024

This PDF was generated from https://docs.netapp.com/us-en/netapp-solutions/ai/ai-protopia-overview.html on March 19, 2024. Always check docs.netapp.com for the latest.

Table of Contents

Responsible AI and confidential inferencing - NetApp AI with Protopia Image Transformation . . . 1
TR-4928: Responsible AI and confidential inferencing - NetApp AI with Protopia Image and Data Transformation . . . 1
Solution areas . . . 3
Technology overview . . . 5
Test and validation plan . . . 9
Test configuration . . . 9
Test procedure . . . 9
Inferencing accuracy comparison . . . 24
Obfuscation speed . . . 25
Conclusion . . . 25
Where to find additional information and acknowledgements . . . 26

Responsible AI and confidential inferencing - NetApp AI with Protopia Image Transformation

TR-4928: Responsible AI and confidential inferencing - NetApp AI with Protopia Image and Data Transformation

Sathish Thyagarajan, Michael Oglesby, NetApp
Byung Hoon Ahn, Jennifer Cwagenberg, Protopia

Visual interpretations have become an integral part of communication with the emergence of image capturing and image processing. Artificial intelligence (AI) in digital image processing brings novel business opportunities, such as in the medical field for cancer and other disease identification, in geospatial visual analytics for studying environmental hazards, in pattern recognition, in video processing for fighting crime, and so on. However, this opportunity also comes with extraordinary responsibilities.

The more decisions organizations put into the hands of AI, the more they accept risks related to data privacy and security and legal, ethical, and regulatory issues. Responsible AI enables a practice that allows companies and government organizations to build trust and governance that is crucial for AI at scale in large enterprises. This document describes an AI inferencing solution validated by NetApp under three different scenarios by using NetApp data management technologies with Protopia data obfuscation software to privatize sensitive data and reduce risks and ethical concerns.

Millions of images are generated every day with various digital devices by both consumers and business entities. The consequent massive explosion of data and computational workload makes businesses turn to cloud computing platforms for scale and efficiency. Meanwhile, privacy concerns over the sensitive information contained in image data arise with transfer to a public cloud. The lack of security and privacy assurances become the main barrier to deployment of image- processing AI systems.

Additionally, there is the right to erasure by the GDPR, the right of an individual to request that an organization erase all their personal data. There is also the Privacy Act, which establishes a code of fair information practices. Digital images such as photographs can constitute personal data under the GDPR, which governs how data must be collected, processed, and erased. Failure to do so is a failure to comply with GDPR, which might lead to hefty fines for breaching compliances that can be seriously damaging to organizations. Privacy principles are among the backbone of implementing responsible AI that ensure fairness in the machine learning (ML) and deep learning (DL) model predictions and lowers risks associated with violating privacy or regulatory compliance.

This document describes a validated design solution under three different scenarios with and without image obfuscation relevant to preserving privacy and deploying a responsible AI solution:

- **Scenario 1.** On-demand inferencing within Jupyter notebook.
- **Scenario 2.** Batch inferencing on Kubernetes.
- **Scenario 3.** NVIDIA Triton inference server.

For this solution, we use the Face Detection Data Set and Benchmark (FDDB), a dataset of face regions designed for studying the problem of unconstrained face detection, combined with the PyTorch machine learning framework for implementation of FaceBoxes. This dataset contains the annotations for 5171 faces in a set of 2845 images of various resolutions. Furthermore, this technical report presents some of the solution areas and relevant use cases gathered from NetApp customers and field engineers in situations where this

solution is applicable.

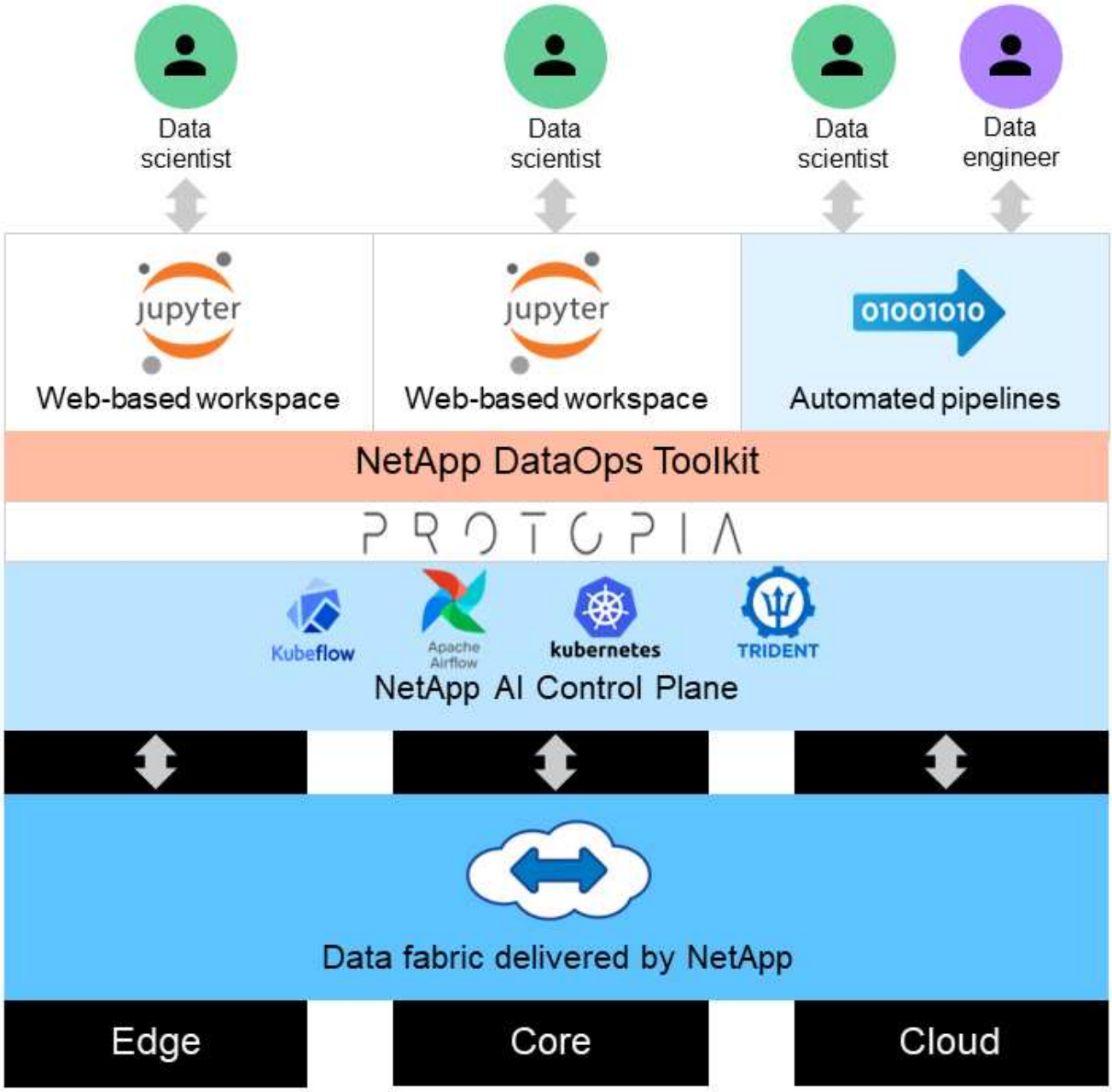
Target audience

This technical report is intended for the following audiences:

- Business leaders and enterprise architects who want to design and deploy responsible AI and address data protection and privacy issues concerning facial image processing in public spaces.
- Data scientists, data engineers, AI/ machine learning (ML) researchers, and developers of AI/ML systems who aim to protect and preserve privacy.
- Enterprise architects who design data obfuscation solutions for AI/ML models and applications that comply with regulatory standards such as GDPR, CCPA, or the Privacy Act of the Department of Defense (DoD) and government organizations.
- Data scientists and AI engineers looking for efficient ways to deploy deep learning (DL) and AI/ML/DL inferencing models that protect sensitive information.
- Edge device managers and edge server administrators responsible for deployment and management of edge inferencing models.

Solution architecture

This solution is designed to handle real-time and batch inferencing AI workloads on large datasets by using the processing power of GPUs alongside traditional CPUs. This validation demonstrates the privacy-preserving inference for ML and optimal data management required for organizations seeking responsible AI deployments. This solution provides an architecture suited for a single or multi-node Kubernetes platform for edge and cloud computing interconnected with NetApp ONTAP AI at the core on-premises, NetApp DataOps Toolkit, and Protopia obfuscation software using Jupyter Lab and CLI interfaces. The following figure shows the logical architecture overview of data fabric powered by NetApp with DataOps Toolkit and Protopia.



Protopia obfuscation software runs seamlessly on top of the NetApp DataOps Toolkit and transforms the data before leaving the storage server.

Solution areas

Digital image processing comes with a lot of advantages, allowing many organizations to make the most of data associated with visual representations. This NetApp and Protopia solution provides a unique AI inferencing design to protect and privatize AI/ML data across the ML/DL life cycle. It enables customers to retain ownership of sensitive data, use public- or hybrid-cloud deployment models for scale and efficiency by alleviating concerns related to privacy, and deploy AI inferencing at the edge.

Environmental intelligence

There are many ways industries can take advantage of geospatial analytics in the areas of environmental hazards. Governments and the department of public works can derive actionable insights on public health and weather conditions to better advise the public during a pandemic or a natural disaster such as wildfires. For example, you can identify a COVID- positive patient in public spaces, such as airports or hospitals, without compromising the privacy of the affected individual and alert the respective authorities and the public in the vicinity for necessary safety measures.

Edge device wearables

In the military and on battlefields, you can use AI inferencing on the edge as wearable devices to track soldier health, monitor driver behavior, and alert authorities on the safety and associated risks of approaching military vehicles while preserving and protecting the privacy of soldiers. The future of the military is going high-tech with the Internet of Battlefield Things (IoBT) and the Internet of Military Things (IoMT) for wearable combat gear that help soldiers identify enemies and perform better in battle by using rapid edge computing. Protecting and preserving visual data collected from edge devices such as drones and wearable gears is crucial to keep hackers and the enemy at bay.

Noncombatant evacuation operations

Noncombatant evacuation operations (NEOs) are conducted by the DoD to assist in evacuating US citizens and nationals, DoD civilian personnel, and designated persons (host nation (HN) and third-country nationals (TCNs)) whose lives are in danger to an appropriate safe haven. The administrative controls in place use largely manual evacuee screening processes. However, the accuracy, security, and speed of evacuee identification, evacuee tracking, and threat screening could potentially be improved by using highly automated AI/ML tools combined with AI/ML video obfuscation technologies.

Healthcare and biomedical research

Image processing is used to diagnose pathologies for surgical planning from 3D images obtained from computed tomography (CT) or magnetic resonance imaging (MRI). HIPAA privacy rules govern how data must be collected, processed, and erased by organizations for all personal information and digital images like photographs. For data to qualify as sharable under the HIPAA Safe Harbor regulations, full-face photographic images and any comparable images must be removed. Automated techniques like de-identification or skull -stripping algorithms used to obscure an individual's facial features from structural CT/MR images have become an essential part of the data sharing process for biomedical research institutions.

Cloud migration of AI/ML analytics

Enterprise customers have traditionally trained and deployed AI/ML models on-premises. For economies of scale and efficiency reasons, these customers are expanding to move AI/ML functions into public, hybrid, or multi-cloud cloud deployments. However, they are bound by what data can be exposed to other infrastructures. NetApp solutions address a full range of cybersecurity threats required for data protection and security assessment and, when combined with Protopia data transformation, minimize the risks associated with migrating image processing AI/ML workloads to the cloud.

For additional use cases for edge computing and AI inferencing across other industries, see TR-4886 AI Inferencing at the Edge and the NetApp AI blog, Intelligence versus privacy.

Technology overview

This section provides an overview of the various technical components required to complete this solution.

Protopia

Protopia AI offers a unobtrusive, software-only solution for confidential inference in the market today. The Protopia solution delivers unparalleled protection for inference services by minimizing exposure of sensitive information. AI is only fed the information in the data record that is truly essential to perform the task at hand and nothing more. Most inference tasks do not use all the information that exists in every data record. Regardless of whether your AI is consuming images, voice, video, or even structured tabular data, Protopia delivers only what the inference service needs. The patented core technology uses mathematically curated noise to stochastically transform the data and garble the information that is not needed by a given ML service. This solution does not mask the data; rather, it changes the data representation by using curated random noise.

The Protopia solution formulates the problem of changing the representation as a gradient-based perturbation maximization method that still retains the pertinent information in the input feature space with respect to the functionality of the model. This discovery process is run as a fine-tuning pass at the end of training the ML model. After the pass automatically generates a set of probability distributions, a low-overhead data transformation applies noise samples from these distributions to the data, obfuscating it before passing it to the model for inferencing.

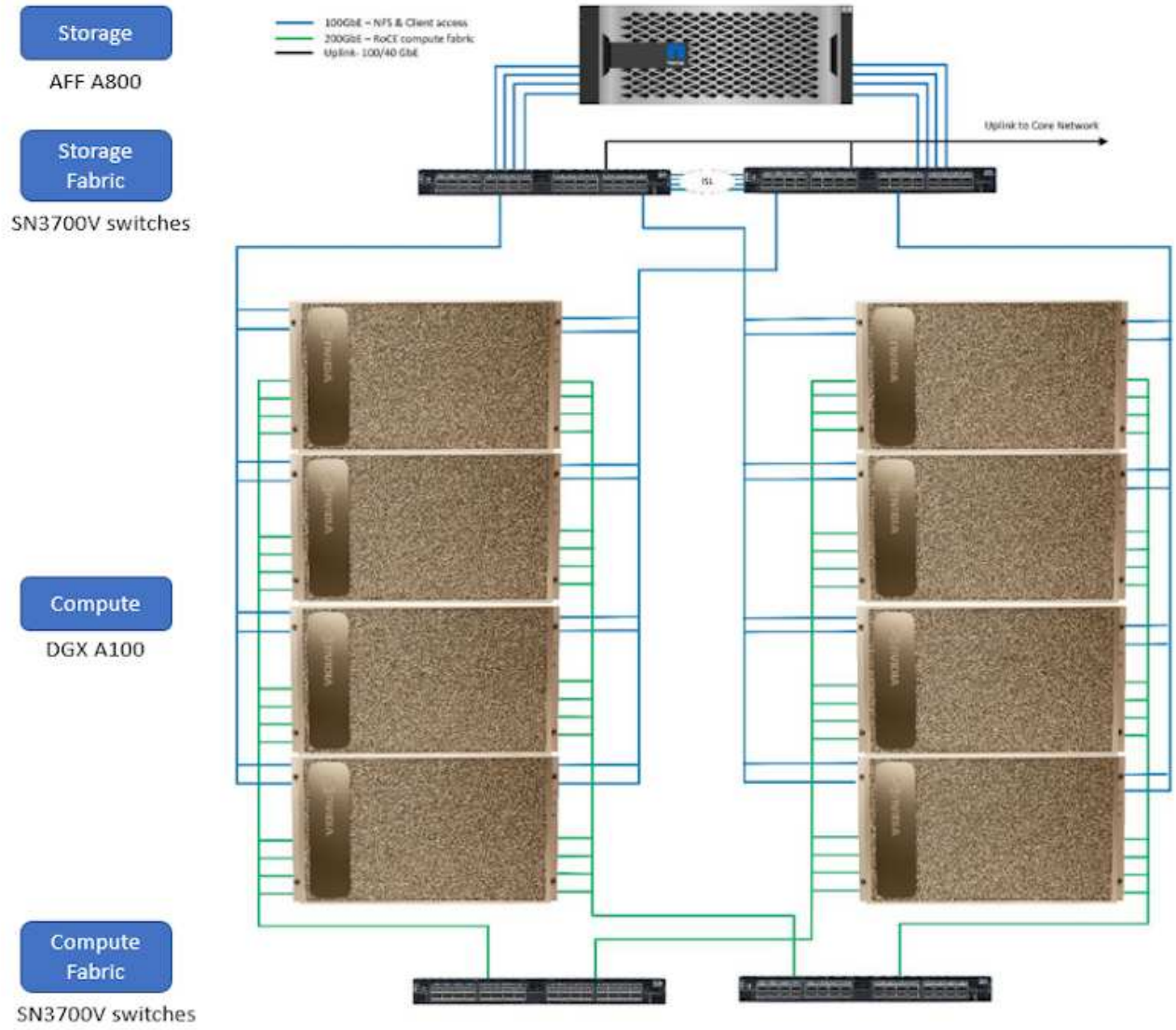
NetApp ONTAP AI

The NetApp ONTAP AI reference architecture, powered by DGX A100 systems and NetApp cloud connected storage systems, was developed and verified by NetApp and NVIDIA. It gives IT organizations an architecture that provides the following benefits:

- Eliminates design complexities
- Allows independent scaling of compute and storage
- Enables customers to start small and scale seamlessly
- Offers a range of storage options for various performance and cost points

ONTAP AI tightly integrates DGX A100 systems and NetApp AFF A800 storage systems with state-of-the-art networking. ONTAP AI simplifies AI deployments by eliminating design complexity and guesswork. Customers can start small and grow nondisruptively while intelligently managing data from the edge to the core to the cloud and back.

The following figure shows several variations in the ONTAP AI family of solutions with DGX A100 systems. AFF A800 system performance is verified with up to eight DGX A100 systems. By adding storage controller pairs to the ONTAP cluster, the architecture can scale to multiple racks to support many DGX A100 systems and petabytes of storage capacity with linear performance. This approach offers the flexibility to alter compute-to-storage ratios independently based on the size of the DL models that are used and the required performance metrics.



For additional information about ONTAP AI, see NVA-1153: NetApp ONTAP AI with NVIDIA DGX A100 Systems and Mellanox Spectrum Ethernet Switches.

NetApp ONTAP

ONTAP 9.11, the latest generation of storage management software from NetApp, enables businesses to modernize infrastructure and transition to a cloud-ready data center. Leveraging industry-leading data management capabilities, ONTAP enables the management and protection of data with a single set of tools, regardless of where that data resides. You can also move data freely to wherever it is needed: the edge, the core, or the cloud. ONTAP 9.11 includes numerous features that simplify data management, accelerate, and protect critical data, and enable next generation infrastructure capabilities across hybrid cloud architectures.

NetApp DataOps Toolkit

NetApp DataOps Toolkit is a Python library that makes it simple for developers, data scientists, DevOps engineers, and data engineers to perform various data management tasks, such as near-instantaneous provisioning of a new data volume or JupyterLab workspace, near-instantaneous cloning of a data volume or JupyterLab workspace, and near-instantaneous taking snapshots of a data volume or JupyterLab workspace for traceability or baselining. This Python library can function as either a command-line utility or a library of

functions that you can import into any Python program or Jupyter notebook.

NVIDIA Triton Inference Server

NVIDIA Triton Inference Server is an open-source inference serving software that helps standardize model deployment and execution to deliver fast and scalable AI in production. Triton Inference Server streamlines AI inferencing by enabling teams to deploy, run, and scale trained AI models from any framework on any GPU- or CPU-based infrastructure. Triton Inference Server supports all major frameworks, such as TensorFlow, NVIDIA TensorRT, PyTorch, MXNet, OpenVINO, and so on. Triton integrates with Kubernetes for orchestration and scaling that you can use in all major public cloud AI and Kubernetes platforms. It's also integrated with many MLOps software solutions.

PyTorch

PyTorch is an open-source ML framework. It is an optimized tensor library for deep learning that uses GPUs and CPUs. The PyTorch package contains data structures for multidimensional tensors that provide many utilities for efficient serializing of tensors among other useful utilities. It also has a CUDA counterpart that enables you to run your tensor computations on an NVIDIA GPU with compute capability. In this validation, we use the OpenCV-Python (cv2) library to validate our model while taking advantage of Python's most intuitive computer vision concepts.

Simplify data management

Data management is crucial to enterprise IT operations and data scientists so that appropriate resources are used for AI applications and training AI/ML datasets. The following additional information about NetApp technologies is out of scope for this validation but might be relevant depending on your deployment.

ONTAP data management software includes the following features to streamline and simplify operations and reduce your total cost of operation:

- Inline data compaction and expanded deduplication. Data compaction reduces wasted space inside storage blocks, and deduplication significantly increases effective capacity. This applies to data stored locally and data tiered to the cloud.
- Minimum, maximum, and adaptive quality of service (AQoS). Granular quality of service (QoS) controls help maintain performance levels for critical applications in highly shared environments.
- NetApp FabricPool. Provides automatic tiering of cold data to public and private cloud storage options, including Amazon Web Services (AWS), Azure, and NetApp StorageGRID storage solution. For more information about FabricPool, see TR-4598: FabricPool best practices.

Accelerate and protect data

ONTAP delivers superior levels of performance and data protection and extends these capabilities in the following ways:

- Performance and lower latency. ONTAP offers the highest possible throughput at the lowest possible latency.
- Data protection. ONTAP provides built-in data protection capabilities with common management across all platforms.
- NetApp Volume Encryption (NVE). ONTAP offers native volume-level encryption with both onboard and External Key Management support.
- Multitenancy and multifactor authentication. ONTAP enables sharing of infrastructure resources with the highest levels of security.

Future-proof infrastructure

ONTAP helps meet demanding and constantly changing business needs with the following features:

- Seamless scaling and nondisruptive operations. ONTAP supports the nondisruptive addition of capacity to existing controllers and to scale-out clusters. Customers can upgrade to the latest technologies, such as NVMe and 32Gb FC, without costly data migrations or outages.
- Cloud connection. ONTAP is the most cloud-connected storage management software, with options for software-defined storage (ONTAP Select) and cloud-native instances (NetApp Cloud Volumes Service) in all public clouds.
- Integration with emerging applications. ONTAP offers enterprise-grade data services for next generation platforms and applications, such as autonomous vehicles, smart cities, and Industry 4.0, by using the same infrastructure that supports existing enterprise apps.

NetApp Astra Control

The NetApp Astra product family offers storage and application-aware data management services for Kubernetes applications on-premises and in the public cloud, powered by NetApp storage and data management technologies. It enables you to easily back up Kubernetes applications, migrate data to a different cluster, and instantly create working application clones. If you need to manage Kubernetes applications running in a public cloud, see the documentation for Astra Control Service. Astra Control Service is a NetApp-managed service that provides application-aware data management of Kubernetes clusters in Google Kubernetes Engine (GKE) and Azure Kubernetes Service (AKS).

NetApp Astra Trident

Astra Trident from NetApp is an open-source dynamic storage orchestrator for Docker and Kubernetes that simplifies the creation, management, and consumption of persistent storage. Trident, a Kubernetes-native application, runs directly within a Kubernetes cluster. Trident enables customers to seamlessly deploy DL container images onto NetApp storage and provides an enterprise-grade experience for AI container deployments. Kubernetes users (ML developers, data scientists, and so on) can create, manage, and automate orchestration and cloning to take advantage of advanced data management capabilities powered by NetApp technology.

NetApp BlueXP Copy and Sync

BlueXP Copy and Sync is a NetApp service for rapid and secure data synchronization. Whether you need to transfer files between on-premises NFS or SMB file shares, NetApp StorageGRID, NetApp ONTAP S3, NetApp Cloud Volumes Service, Azure NetApp Files, Amazon Simple Storage Service (Amazon S3), Amazon Elastic File System (Amazon EFS), Azure Blob, Google Cloud Storage, or IBM Cloud Object Storage, BlueXP Copy and Sync moves the files where you need them quickly and securely. After your data is transferred, it is fully available for use on both source and target. BlueXP Copy and Syncc continuously synchronizes the data based on your predefined schedule, moving only the deltas, so that time and money spent on data replication is minimized. BlueXP Copy and Sync is a software-as-a-service (SaaS) tool that is extremely simple to set up and use. Data transfers that are triggered by BlueXP Copy and Sync are carried out by data brokers. You can deploy BlueXP Copy and Sync data brokers in AWS, Azure, Google Cloud Platform, or on-premises.

NetApp BlueXP Classification

Driven by powerful AI algorithms, NetApp BlueXP Classification provides automated controls and data governance across your entire data estate. You can easily pinpoint cost-savings, identify compliance and privacy concerns, and find optimization opportunities. The BlueXP Classification dashboard gives you the insight to identify duplicate data to eliminate redundancy, map personal, nonpersonal, and sensitive data and

turn on alerts for sensitive data and anomalies.

Test and validation plan

For this solution design, the following three scenarios were validated:

- An inferencing task, with and without Protopia obfuscation, within a JupyterLab workspace that was orchestrated by using the NetApp DataOps Toolkit for Kubernetes.
- A batch inferencing job, with and without Protopia obfuscation, on Kubernetes with a data volume that was orchestrated by using NetApp DataOps Toolkit for Kubernetes.
- An inferencing task using an NVIDIA Triton Inference Server instance that was orchestrated by using the NetApp DataOps Toolkit for Kubernetes. We applied Protopia obfuscation to the image before invoking the Triton inference API to simulate the common requirement that any data that is transmitted over the network must be obfuscated. This workflow is applicable to use cases where data is collected within a trusted zone but must be passed outside of that trusted zone for inferencing. Without Protopia obfuscation, it is not possible to implement this type of workflow without sensitive data leaving the trusted zone.

Test configuration

The following table outlines the solution design validation environment.

Component	Version
Kubernetes	1.21.6
NetApp Astra Trident CSI Driver	22.01.0
NetApp DataOps Toolkit for Kubernetes	2.3.0
NVIDIA Triton Inference Server	21.11-py3

Test procedure

This section describes the tasks needed to complete the validation.

Prerequisites

To execute the tasks outlined in this section, you must have access to a Linux or macOS host with the following tools installed and configured:

- Kubectl (configured for access to an existing Kubernetes cluster)
  - Installation and configuration instructions can be found here.
- NetApp DataOps Toolkit for Kubernetes
  - Installation instructions can be found here.

Scenario 1 – On-demand inferencing in JupyterLab

1. Create a Kubernetes namespace for AI/ML inferencing workloads.

```
$ kubectl create namespace inference
namespace/inference created
```

2. Use the NetApp DataOps Toolkit to provision a persistent volume for storing the data on which you will perform the inferencing.

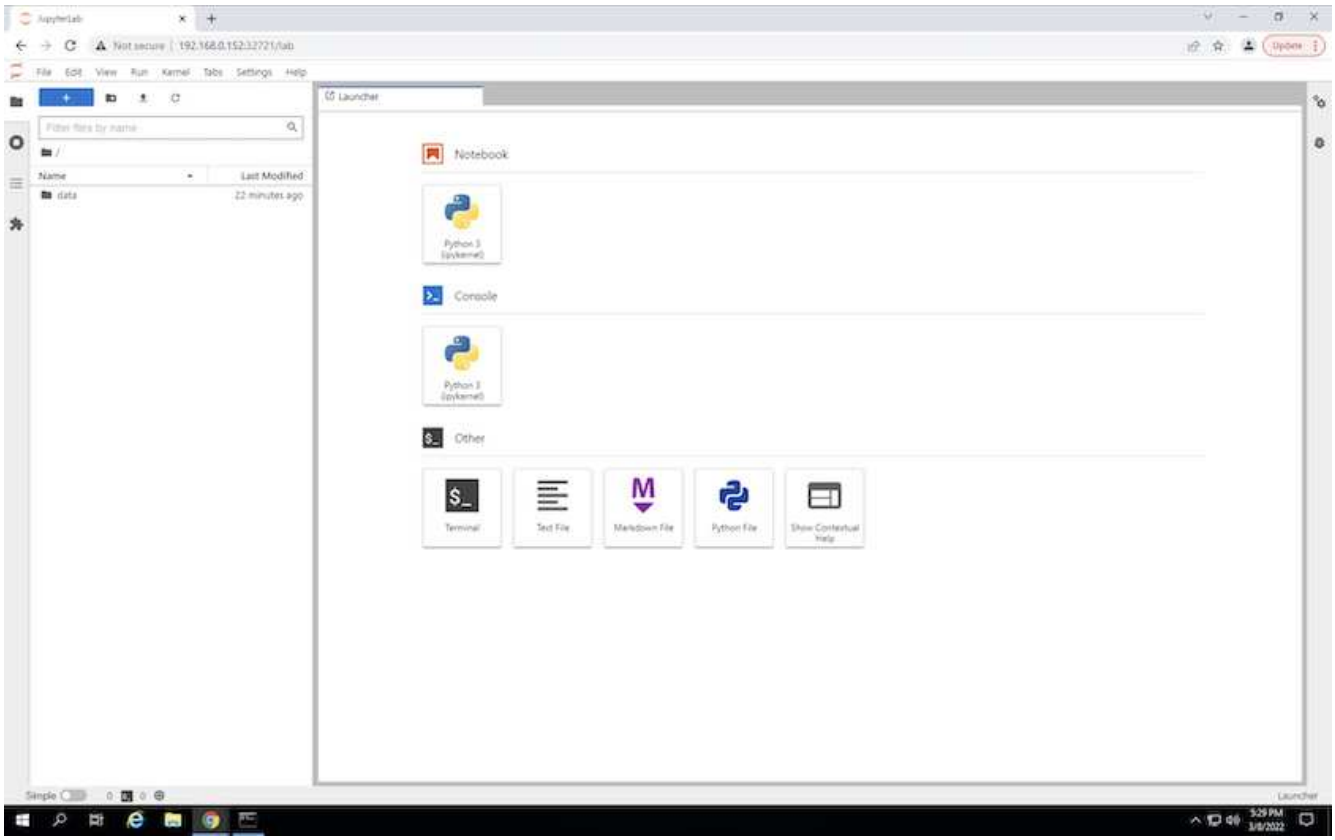
```
$ netapp_dataops_k8s_cli.py create volume --namespace=inference --pvc
-name=inference-data --size=50Gi
Creating PersistentVolumeClaim (PVC) 'inference-data' in namespace
'inference'.
PersistentVolumeClaim (PVC) 'inference-data' created. Waiting for
Kubernetes to bind volume to PVC.
Volume successfully created and bound to PersistentVolumeClaim (PVC)
'inference-data' in namespace 'inference'.
```

3. Use the NetApp DataOps Toolkit to create a new JupyterLab workspace. Mount the persistent volume that was created in the previous step by using the `--mount- pvc` option. Allocate NVIDIA GPUs to the workspace as necessary by using the `-- nvidia-gpu` option.

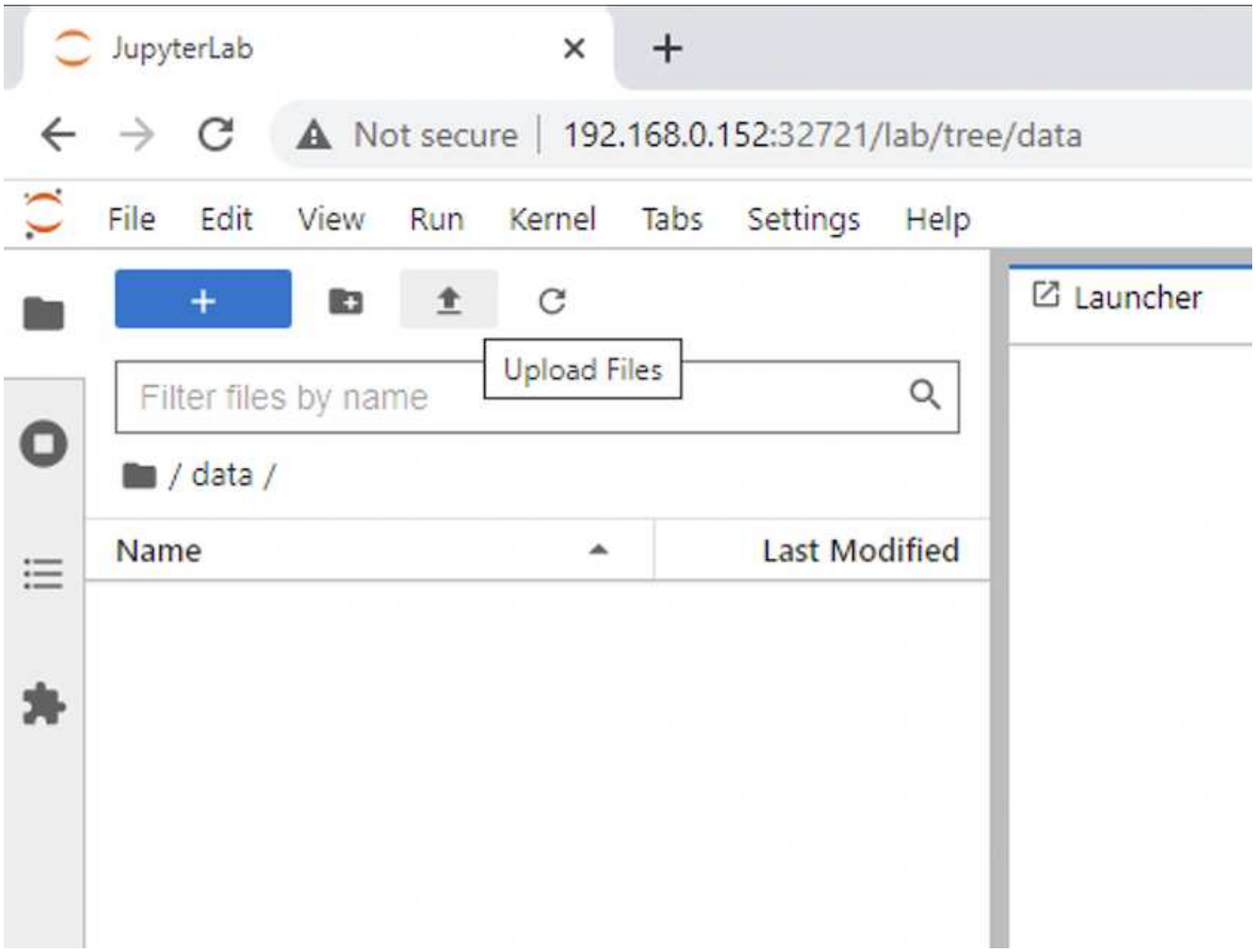
   In the following example, the persistent volume `inference-data` is mounted to the JupyterLab workspace container at `/home/jovyan/data`. When using official Project Jupyter container images, `/home/jovyan` is presented as the top-level directory within the JupyterLab web interface.

```
$ netapp_dataops_k8s_cli.py create jupyterlab --namespace=inference
--workspace-name=live-inference --size=50Gi --nvidia-gpu=2 --mount
-pvc=inference-data:/home/jovyan/data
Set workspace password (this password will be required in order to
access the workspace):
Re-enter password:
Creating persistent volume for workspace...
Creating PersistentVolumeClaim (PVC) 'ntap-dsutil-jupyterlab-live-
inference' in namespace 'inference'.
PersistentVolumeClaim (PVC) 'ntap-dsutil-jupyterlab-live-inference'
created. Waiting for Kubernetes to bind volume to PVC.
Volume successfully created and bound to PersistentVolumeClaim (PVC)
'ntap-dsutil-jupyterlab-live-inference' in namespace 'inference'.
Creating Service 'ntap-dsutil-jupyterlab-live-inference' in namespace
'inference'.
Service successfully created.
Attaching Additional PVC: 'inference-data' at mount_path:
'/home/jovyan/data'.
Creating Deployment 'ntap-dsutil-jupyterlab-live-inference' in namespace
'inference'.
Deployment 'ntap-dsutil-jupyterlab-live-inference' created.
Waiting for Deployment 'ntap-dsutil-jupyterlab-live-inference' to reach
Ready state.
Deployment successfully created.
Workspace successfully created.
To access workspace, navigate to http://192.168.0.152:32721
```

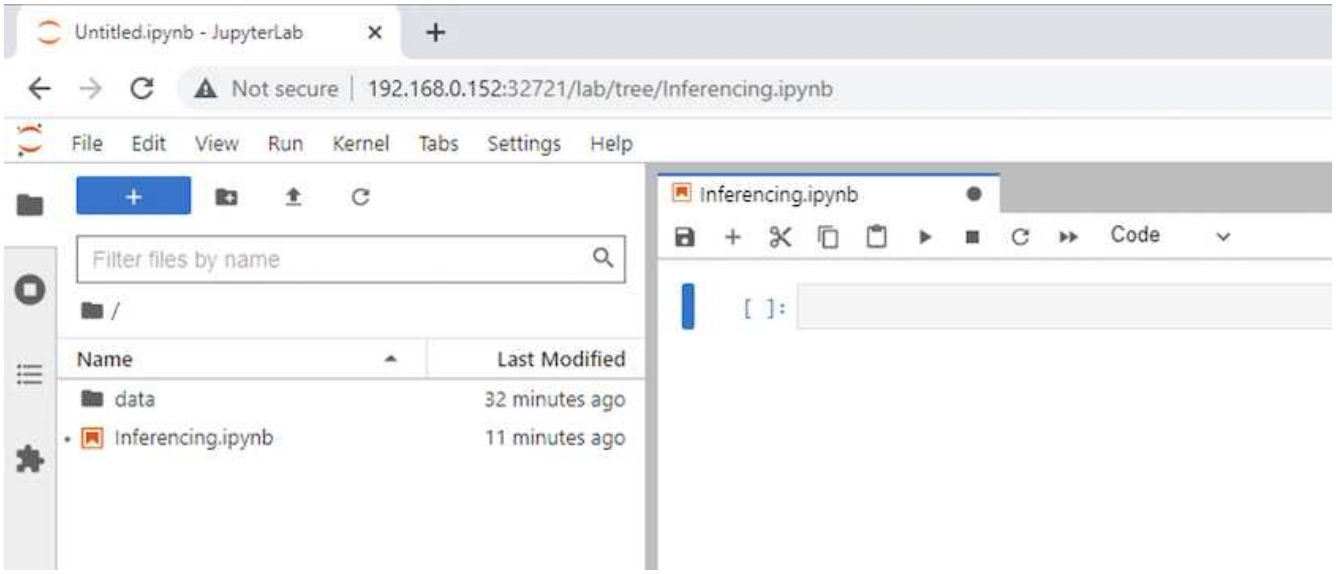
4. Access the JupyterLab workspace by using the URL specified in the output of the `create jupyterlab` command. The data directory represents the persistent volume that was mounted to the workspace.



5. Open the `data` directory and upload the files on which the inferencing is to be performed. When files are uploaded to the data directory, they are automatically stored on the persistent volume that was mounted to the workspace. To upload files, click the Upload Files icon, as shown in the following image.



6. Return to the top-level directory and create a new notebook.



7. Add inferencing code to the notebook. The following example shows inferencing code for an image detection use case.

Launcher | image-demo-pytorch.ipynb

Code | Python 3 (ipykernel)

STEP 3-1: Clean (Without obfuscation) detection

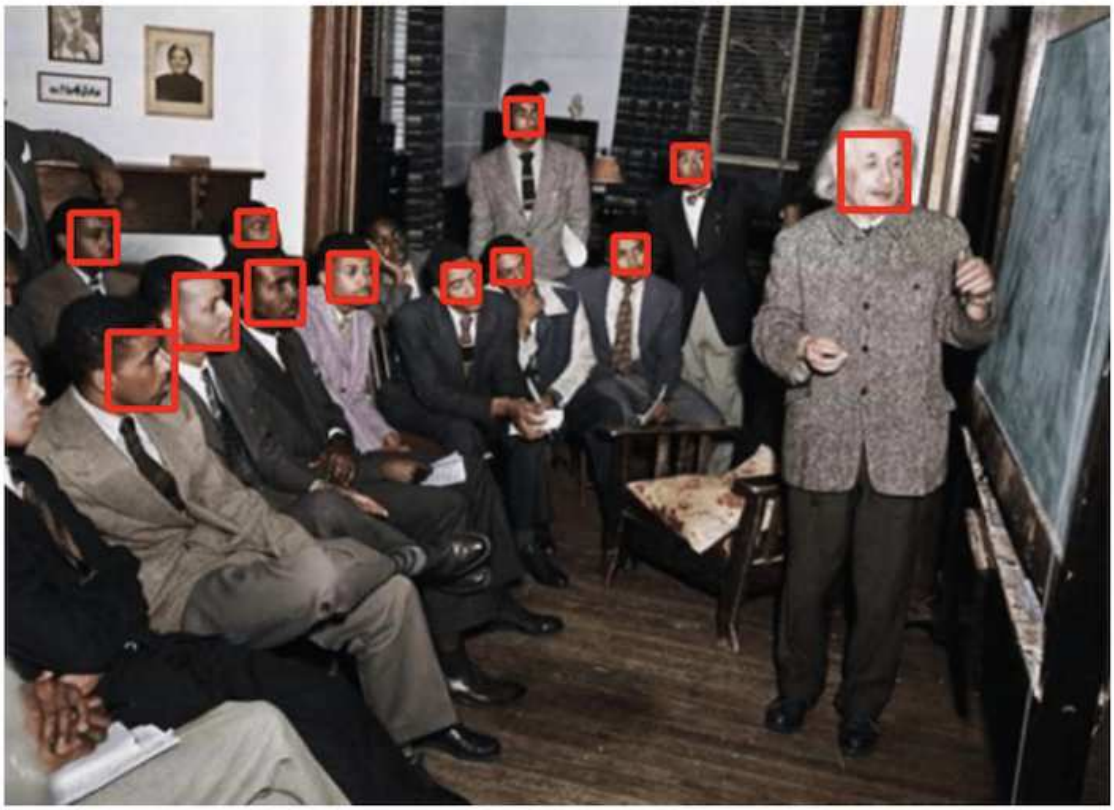
[9]:
```
# get current frame
frame = input_image

# preprocess input
preprocessed_input = preprocess_input(frame)
preprocessed_input = torch.Tensor(preprocessed_input).to(device)

# run forward pass
clean_activation = clean_model.forward_head(preprocessed_input)  # runs the first few layers
loc, pred = clean_model.forward_tail(clean_activation)  # runs rest of the layers

# postprocess output
clean_pred = (loc.detach().cpu().numpy(), pred.detach().cpu().numpy())
clean_outputs = postprocess_outputs(
    clean_pred, [[input_image_width, input_image_height]], priors, THRESHOLD
)

# draw rectangles
clean_frame = copy.deepcopy(frame)  # needs to be deep copy
for (x1, y1, x2, y2, s) in clean_outputs[0]:
    x1, y1 = int(x1), int(y1)
    x2, y2 = int(x2), int(y2)
    cv2.rectangle(clean_frame, (x1, y1), (x2, y2), (0, 0, 255), 4)
```

Launcher | image-demo-pytorch.ipynb

Code | Python 3 (ipykernel)

Visualize Clean (Without obfuscation) detection

[10]:
```
show_cv2_image(clean_frame, scale=2)
```



8. Add Protopia obfuscation to your inferencing code. Protopia works directly with customers to provide use-case specific documentation and is outside of the scope of this technical report. The following example shows inferencing code for an image detection use case with Protopia obfuscation added.

Launcher | image-demo-pytorch.ipynb

Code | Python 3 (ipykernel)

STEP 3-2: Protopia AI (With obfuscation) detection

[11]:
```
# get current frame
frame = input_image

# preprocess input
preprocessed_input = preprocess_input(frame)
preprocessed_input = torch.Tensor(preprocessed_input).to(device)

# run forward pass
not_noisy_activation = noisy_model.forward_head(preprocessed_input)  # runs the first few layers
##################################################################
#          SINGLE ADITIONAL LINE FOR PRIVATE INFERENCE           #
##################################################################
noisy_activation = noisy_model.forward_noise(not_noisy_activation)
##################################################################
loc, pred = noisy_model.forward_tail(noisy_activation)  # runs rest of the layers

# postprocess output
noisy_pred = (loc.detach().cpu().numpy(), pred.detach().cpu().numpy())
noisy_outputs = postprocess_outputs(
    noisy_pred, [[input_image_width, input_image_height]], priors, THRESHOLD * 0.5
)

# get reconstruction of the noisy activation
noisy_reconstruction = decoder_function(noisy_activation)
noisy_reconstruction = noisy_reconstruction.detach().cpu().numpy()[0]
noisy_reconstruction = unpreprocess_output(
    noisy_reconstruction, (input_image_width, input_image_height), True
).astype(np.uint8)

# draw rectangles
for (x1, y1, x2, y2, s) in noisy_outputs[0]:
    x1, y1 = int(x1), int(y1)
    x2, y2 = int(x2), int(y2)
    cv2.rectangle(noisy_reconstruction, (x1, y1), (x2, y2), (0, 0, 255), 4)
```

Launcher | image-demo-pytorch.ipynb

Code | Python 3 (ipykernel)

Visualize Protopia AI (With obfuscation) detection

[12]:
```
show_cv2_image(noisy_reconstruction, scale=2)
```

Scenario 2 – Batch inferencing on Kubernetes

1. Create a Kubernetes namespace for AI/ML inferencing workloads.

```
$ kubectl create namespace inference
namespace/inference created
```

2. Use the NetApp DataOps Toolkit to provision a persistent volume for storing the data on which you will perform the inferencing.

```
$ netapp_dataops_k8s_cli.py create volume --namespace=inference --pvc
-name=inference-data --size=50Gi
Creating PersistentVolumeClaim (PVC) 'inference-data' in namespace
'inference'.
PersistentVolumeClaim (PVC) 'inference-data' created. Waiting for
Kubernetes to bind volume to PVC.
Volume successfully created and bound to PersistentVolumeClaim (PVC)
'inference-data' in namespace 'inference'.
```

3. Populate the new persistent volume with the data on which you will perform the inferencing.

   There are several methods for loading data onto a PVC. If your data is currently stored in an S3-compatible object storage platform, such as NetApp StorageGRID or Amazon S3, then you can use NetApp DataOps Toolkit S3 Data Mover capabilities. Another simple method is to create a JupyterLab workspace and then upload files through the JupyterLab web interface, as outlined in Steps 3 to 5 in the section "Scenario 1 – On-demand inferencing in JupyterLab."

4. Create a Kubernetes job for your batch inferencing task. The following example shows a batch inferencing job for an image detection use case. This job performs inferencing on each image in a set of images and writes inferencing accuracy metrics to stdout.

```
$ vi inference-job-raw.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: netapp-inference-raw
  namespace: inference
spec:
  backoffLimit: 5
  template:
    spec:
      volumes:
      - name: data
        persistentVolumeClaim:
          claimName: inference-data
      - name: dshm
        emptyDir:
          medium: Memory
      containers:
      - name: inference
        image: netapp-protopia-inference:latest
        imagePullPolicy: IfNotPresent
        command: ["python3", "run-accuracy-measurement.py", "--dataset",
"/data/netapp-face-detection/FDDB"]
        resources:
          limits:
            nvidia.com/gpu: 2
        volumeMounts:
        - mountPath: /data
          name: data
        - mountPath: /dev/shm
          name: dshm
      restartPolicy: Never
$ kubectl create -f inference-job-raw.yaml
job.batch/netapp-inference-raw created
```

5. Confirm that the inferencing job completed successfully.

```
$ kubectl -n inference logs netapp-inference-raw-255sp
100%|██████████| 89/89 [00:52<00:00,  1.68it/s]
Reading Predictions : 100%|██████████| 10/10 [00:01<00:00,  6.23it/s]
Predicting ... : 100%|██████████| 10/10 [00:16<00:00,  1.64s/it]
==================== Results ====================
FDDB-fold-1 Val AP: 0.9491256561145955
FDDB-fold-2 Val AP: 0.9205024466101926
FDDB-fold-3 Val AP: 0.9253013871078468
FDDB-fold-4 Val AP: 0.9399781485863011
FDDB-fold-5 Val AP: 0.9504280149478732
FDDB-fold-6 Val AP: 0.9416473519339292
FDDB-fold-7 Val AP: 0.9241631566241117
FDDB-fold-8 Val AP: 0.9072663297546659
FDDB-fold-9 Val AP: 0.9339648715035469
FDDB-fold-10 Val AP: 0.9447707905560152
FDDB Dataset Average AP: 0.9337148153739079
==================================================
mAP: 0.9337148153739079
```

6. Add Protopia obfuscation to your inferencing job. You can find use case-specific instructions for adding Protopia obfuscation directly from Protopia, which is outside of the scope of this technical report. The following example shows a batch inferencing job for a face detection use case with Protopia obfuscation added by using an ALPHA value of 0.8. This job applies Protopia obfuscation before performing inferencing for each image in a set of images and then writes inferencing accuracy metrics to stdout.

   We repeated this step for ALPHA values 0.05, 0.1, 0.2, 0.4, 0.6, 0.8, 0.9, and 0.95. You can see the results in “Inferencing accuracy comparison.”

```
$ vi inference-job-protopia-0.8.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: netapp-inference-protopia-0.8
  namespace: inference
spec:
  backoffLimit: 5
  template:
    spec:
      volumes:
      - name: data
        persistentVolumeClaim:
          claimName: inference-data
      - name: dshm
        emptyDir:
          medium: Memory
      containers:
      - name: inference
        image: netapp-protopia-inference:latest
        imagePullPolicy: IfNotPresent
        env:
        - name: ALPHA
          value: "0.8"
        command: ["python3", "run-accuracy-measurement.py", "--dataset",
"/data/netapp-face-detection/FDDB", "--alpha", "$(ALPHA)", "--noisy"]
        resources:
          limits:
            nvidia.com/gpu: 2
        volumeMounts:
        - mountPath: /data
          name: data
        - mountPath: /dev/shm
          name: dshm
      restartPolicy: Never
$ kubectl create -f inference-job-protopia-0.8.yaml
job.batch/netapp-inference-protopia-0.8 created
```

7. Confirm that the inferencing job completed successfully.

```
$ kubectl -n inference logs netapp-inference-protopia-0.8-b4dkz
100%|██████████| 89/89 [01:05<00:00,  1.37it/s]
Reading Predictions : 100%|██████████| 10/10 [00:02<00:00,  3.67it/s]
Predicting ... : 100%|██████████| 10/10 [00:22<00:00,  2.24s/it]
==================== Results ====================
FDDB-fold-1 Val AP: 0.8953066115834589
FDDB-fold-2 Val AP: 0.8819580264029936
FDDB-fold-3 Val AP: 0.8781107458462862
FDDB-fold-4 Val AP: 0.9085731346308461
FDDB-fold-5 Val AP: 0.9166445508275378
FDDB-fold-6 Val AP: 0.9101178994188819
FDDB-fold-7 Val AP: 0.8383443678423771
FDDB-fold-8 Val AP: 0.8476311547659464
FDDB-fold-9 Val AP: 0.8739624502111121
FDDB-fold-10 Val AP: 0.8905468076424851
FDDB Dataset Average AP: 0.8841195749171925
==================================================
mAP: 0.8841195749171925
```

Scenario 3 – NVIDIA Triton Inference Server

1. Create a Kubernetes namespace for AI/ML inferencing workloads.

```
$ kubectl create namespace inference
namespace/inference created
```

2. Use the NetApp DataOps Toolkit to provision a persistent volume to use as a model repository for the NVIDIA Triton Inference Server.

```
$ netapp_dataops_k8s_cli.py create volume --namespace=inference --pvc
-name=triton-model-repo --size=100Gi
Creating PersistentVolumeClaim (PVC) 'triton-model-repo' in namespace
'inference'.
PersistentVolumeClaim (PVC) 'triton-model-repo' created. Waiting for
Kubernetes to bind volume to PVC.
Volume successfully created and bound to PersistentVolumeClaim (PVC)
'triton-model-repo' in namespace 'inference'.
```

3. Store your model on the new persistent volume in a format that is recognized by the NVIDIA Triton Inference Server.

   There are several methods for loading data onto a PVC. A simple method is to create a JupyterLab workspace and then upload files through the JupyterLab web interface, as outlined in steps 3 to 5 in "Scenario 1 – On-demand inferencing in JupyterLab. "

4. Use NetApp DataOps Toolkit to deploy a new NVIDIA Triton Inference Server instance.

```
$ netapp_dataops_k8s_cli.py create triton-server --namespace=inference
--server-name=netapp-inference --model-repo-pvc-name=triton-model-repo
Creating Service 'ntap-dsutil-triton-netapp-inference' in namespace
'inference'.
Service successfully created.
Creating Deployment 'ntap-dsutil-triton-netapp-inference' in namespace
'inference'.
Deployment 'ntap-dsutil-triton-netapp-inference' created.
Waiting for Deployment 'ntap-dsutil-triton-netapp-inference' to reach
Ready state.
Deployment successfully created.
Server successfully created.
Server endpoints:
http: 192.168.0.152: 31208
grpc: 192.168.0.152: 32736
metrics: 192.168.0.152: 30009/metrics
```

5. Use a Triton client SDK to perform an inferencing task. The following Python code excerpt uses the Triton Python client SDK to perform an inferencing task for an face detection use case. This example calls the Triton API and passes in an image for inferencing. The Triton Inference Server then receives the request, invokes the model, and returns the inferencing output as part of the API results.

```
# get current frame
frame = input_image
# preprocess input
preprocessed_input = preprocess_input(frame)
preprocessed_input = torch.Tensor(preprocessed_input).to(device)
# run forward pass
clean_activation = clean_model_head(preprocessed_input)  # runs the
first few layers
#######################################################################
##############
#          pass clean image to Triton Inference Server API for
inferencing           #
#######################################################################
##############
triton_client =
httpclient.InferenceServerClient(url="192.168.0.152:31208",
verbose=False)
model_name = "face_detection_base"
inputs = []
outputs = []
inputs.append(httpclient.InferInput("INPUT__0", [1, 128, 32, 32],
```

```
"FP32"))
inputs[0].set_data_from_numpy(clean_activation.detach().cpu().numpy(),
binary_data=False)
outputs.append(httpclient.InferRequestedOutput("OUTPUT__0",
binary_data=False))
outputs.append(httpclient.InferRequestedOutput("OUTPUT__1",
binary_data=False))
results = triton_client.infer(
    model_name,
    inputs,
    outputs=outputs,
    #query_params=query_params,
    headers=None,
    request_compression_algorithm=None,
    response_compression_algorithm=None)
#print(results.get_response())
statistics =
triton_client.get_inference_statistics(model_name=model_name,
headers=None)
print(statistics)
if len(statistics["model_stats"]) != 1:
    print("FAILED: Inference Statistics")
    sys.exit(1)

loc_numpy = results.as_numpy("OUTPUT__0")
pred_numpy = results.as_numpy("OUTPUT__1")
#######################################################################
##############
# postprocess output
clean_pred = (loc_numpy, pred_numpy)
clean_outputs = postprocess_outputs(
    clean_pred, [[input_image_width, input_image_height]], priors,
THRESHOLD
)
# draw rectangles
clean_frame = copy.deepcopy(frame)  # needs to be deep copy
for (x1, y1, x2, y2, s) in clean_outputs[0]:
    x1, y1 = int(x1), int(y1)
    x2, y2 = int(x2), int(y2)
    cv2.rectangle(clean_frame, (x1, y1), (x2, y2), (0, 0, 255), 4)
```

6. Add Protopia obfuscation to your inferencing code. You can find use case-specific instructions for adding Protopia obfuscation directly from Protopia; however, this process is outside the scope of this technical report. The following example shows the same Python code that is shown in the preceding step 5, but with Protopia obfuscation added.

   Note that the Protopia obfuscation is applied to the image before it is passed to the Triton API. Thus, the

non-obfuscated image never leaves the local machine. Only the obfuscated image is passed across the network. This workflow is applicable to use cases in which data is collected within a trusted zone but then needs to be passed outside of that trusted zone for inferencing. Without Protopia obfuscation, it is not possible to implement this type of workflow without sensitive data ever leaving the trusted zone.

```
# get current frame
frame = input_image
# preprocess input
preprocessed_input = preprocess_input(frame)
preprocessed_input = torch.Tensor(preprocessed_input).to(device)
# run forward pass
not_noisy_activation = noisy_model_head(preprocessed_input)  # runs the
first few layers
#################################################################
#          obfuscate image locally prior to inferencing          #
#          SINGLE ADITIONAL LINE FOR PRIVATE INFERENCE           #
#################################################################
noisy_activation = noisy_model_noise(not_noisy_activation)
#################################################################
#######################################################################
####################
#          pass obfuscated image to Triton Inference Server API for
inferencing           #
#######################################################################
####################
triton_client =
httpclient.InferenceServerClient(url="192.168.0.152:31208",
verbose=False)
model_name = "face_detection_noisy"
inputs = []
outputs = []
inputs.append(httpclient.InferInput("INPUT__0", [1, 128, 32, 32],
"FP32"))
inputs[0].set_data_from_numpy(noisy_activation.detach().cpu().numpy(),
binary_data=False)
outputs.append(httpclient.InferRequestedOutput("OUTPUT__0",
binary_data=False))
outputs.append(httpclient.InferRequestedOutput("OUTPUT__1",
binary_data=False))
results = triton_client.infer(
    model_name,
    inputs,
    outputs=outputs,
    #query_params=query_params,
    headers=None,
    request_compression_algorithm=None,
```

```
    response_compression_algorithm=None)
#print(results.get_response())
statistics =
triton_client.get_inference_statistics(model_name=model_name,
headers=None)
print(statistics)
if len(statistics["model_stats"]) != 1:
    print("FAILED: Inference Statistics")
    sys.exit(1)

loc_numpy = results.as_numpy("OUTPUT__0")
pred_numpy = results.as_numpy("OUTPUT__1")
#######################################################################
###################

# postprocess output
noisy_pred = (loc_numpy, pred_numpy)
noisy_outputs = postprocess_outputs(
    noisy_pred, [[input_image_width, input_image_height]], priors,
THRESHOLD * 0.5
)
# get reconstruction of the noisy activation
noisy_reconstruction = decoder_function(noisy_activation)
noisy_reconstruction = noisy_reconstruction.detach().cpu().numpy()[0]
noisy_reconstruction = unpreprocess_output(
    noisy_reconstruction, (input_image_width, input_image_height), True
).astype(np.uint8)
# draw rectangles
for (x1, y1, x2, y2, s) in noisy_outputs[0]:
    x1, y1 = int(x1), int(y1)
    x2, y2 = int(x2), int(y2)
    cv2.rectangle(noisy_reconstruction, (x1, y1), (x2, y2), (0, 0, 255),
4)
```

Inferencing accuracy comparison

For this validation, we performed inferencing for an image detection use case by using a set of raw images. We then performed the same inferencing task on the same set of images with Protopia obfuscation added before inferencing. We repeated the task using different values of ALPHA for the Protopia obfuscation component. In the context of Protopia obfuscation, the ALPHA value represents the amount of obfuscation that is applied, with a higher ALPHA value representing a higher level of obfuscation. We then compared inferencing accuracy across these different runs.

The following two tables provide details about our use case and outline the results.

Protopia works directly with customers to determine the appropriate ALPHA value for a specific use case.

Component	Details
Model	FaceBoxes (PyTorch) -
Dataset	FDDB dataset

Protopia obfuscation	ALPHA	Accuracy
No	N/A	0.9337148153739079
Yes	0.05	0.9028766627325002
Yes	0.1	0.9024301009661478
Yes	0.2	0.9081836283186224
Yes	0.4	0.9073066107482036
Yes	0.6	0.8847816568680239
Yes	0.8	0.8841195749171925
Yes	0.9	0.8455427675252052
Yes	0.95	0.8455427675252052

Obfuscation speed

For this validation, we applied Protopia obfuscation to a 1920 x 1080 pixel image five times and measured the amount of time that it took for the obfuscation step to complete each time.

We used PyTorch running on a single NVIDIA V100 GPU to apply the obfuscation, and we cleared the GPU cache between runs. The obfuscation step took 5.47ms, 5.27ms, 4.54ms, 5.24ms, and 4.84ms respectively to complete across the five runs. The average speed was 5.072ms.

Conclusion

Data exists in three states: at rest, in transit, and in compute. An important part of any AI inferencing service should be the protection of data from threats during the entire process. Protecting data during inferencing is critical because the process can expose private information about both external customers and the business providing the inferencing service. Protopia AI is a nonobtrusive software-only solution for confidential AI inferencing in today's market. With Protopia, AI is fed only the transformed information in the data records that is essential to carrying out the AI/ML task at hand and nothing more. This stochastic transformation is not a form of masking and is based on mathematically changing the representation of the data by using curated noise.

NetApp storage systems with ONTAP capabilities deliver the same or better performance as local SSD storage and, combined with the NetApp DataOps Toolkit, offer the following benefits to data scientists, data engineers, AI/ML developers, and business or enterprise IT decision makers:

- Effortless sharing of data between AI systems, analytics, and other critical business systems. This data sharing reduces infrastructure overhead, improves performance, and streamlines data management across the enterprise.
- Independently scalable compute and storage to minimize costs and improve resource usage.
- Streamlined development and deployment workflows using integrated Snapshot copies and clones for instantaneous and space-efficient user workspaces, integrated version control, and automated deployment.
- Enterprise-grade data protection and data governance for disaster recovery, business continuity, and regulatory requirements.
- Simplified invocation of data management operations; rapidly take Snapshot copies of data scientist workspaces for backup and traceability from the NetApp DataOps Toolkit in Jupyter notebooks.

The NetApp and Protopia solution provides a flexible, scale-out architecture that is ideal for enterprise-grade AI inference deployments. It enables data protection and provides privacy for sensitive information where confidential AI inferencing requirements can be met with responsible AI practices in both on-premises and hybrid cloud deployments.

Where to find additional information and acknowledgements

To learn more about the information described in this document, refer to the following documents and/or websites:

- NetApp ONTAP data management software — ONTAP information library

  http://mysupport.netapp.com/documentation/productlibrary/index.html?productID=62286

- NetApp Persistent Storage for Containers—NetApp Trident

  https://netapp.io/persistent-storage-provisioner-for-kubernetes/

- NetApp DataOps Toolkit

  https://github.com/NetApp/netapp-dataops-toolkit

- NetApp Persistent Storage for Containers—NetApp Astra Trident

  https://netapp.io/persistent-storage-provisioner-for-kubernetes/

- Protopia AI—Confidential Inference

  https://protopia.ai/blog/protopia-ai-takes-on-the-missing-link-in-ai-privacy-confidential-inference/

- NetApp BlueXP Copy and Sync

  https://docs.netapp.com/us-en/occm/concept_cloud_sync.html#how-cloud-sync-works

- NVIDIA Triton Inference Server

  https://developer.nvidia.com/nvidia-triton-inference-server

- NVIDIA Triton Inference Server Documentation

https://docs.nvidia.com/deeplearning/triton-inference-server/index.html

- FaceBoxes in PyTorch

  https://github.com/zisianw/FaceBoxes.PyTorch

Acknowledgments

- Mark Cates, Principal Product Manager, NetApp
- Sufian Ahmad, Technical Marketing Engineer, NetApp
- Hadi Esmaeilzadeh, Chief Technology Officer and Professor, Protopia AI

Copyright information

Copyright © 2024 NetApp, Inc. All Rights Reserved. Printed in the U.S. No part of this document covered by copyright may be reproduced in any form or by any means—graphic, electronic, or mechanical, including photocopying, recording, taping, or storage in an electronic retrieval system—without prior written permission of the copyright owner.

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

THIS SOFTWARE IS PROVIDED BY NETAPP “AS IS” AND WITHOUT ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE, WHICH ARE HEREBY DISCLAIMED. IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice. NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp. The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S. patents, foreign patents, or pending applications.

LIMITED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (b)(3) of the Rights in Technical Data -Noncommercial Items at DFARS 252.227-7013 (FEB 2014) and FAR 52.227-19 (DEC 2007).

Data contained herein pertains to a commercial product and/or commercial service (as defined in FAR 2.101) and is proprietary to NetApp, Inc. All NetApp technical data and computer software provided under this Agreement is commercial in nature and developed solely at private expense. The U.S. Government has a non-exclusive, non-transferrable, nonsublicensable, worldwide, limited irrevocable license to use the Data only in connection with and in support of the U.S. Government contract under which the Data was delivered. Except as provided herein, the Data may not be used, disclosed, reproduced, modified, performed, or displayed without the prior written approval of NetApp, Inc. United States Government license rights for the Department of Defense are limited to those rights identified in DFARS clause 252.227-7015(b) (FEB 2014).

Trademark information

NETAPP, the NETAPP logo, and the marks listed at http://www.netapp.com/TM are trademarks of NetApp, Inc. Other company and product names may be trademarks of their respective owners.